Лабораторная работа №7
Команды безусловного и условного переходов в Nasm. Программирование ветвлений.

Студент: Аньсяоянь
Номер студенческого билета: 1132258191
Группа: НПИбд-01-25
Дата выполнения: 26.03.2026

---

1. Цель работы

Изучение команд условного и безусловного переходов, приобретение навыков написания программ с использованием переходов, знакомство с назначением и структурой файла листинга.

---

2. Теоретическое введение

2.1. Команды безусловного перехода

Безусловный переход выполняется инструкцией `jmp` (от англ. jump – прыжок):

```assembly
jmp <адрес_перехода>
```

Адрес перехода может быть меткой, адресом области памяти или именем регистра.

2.2. Регистр флагов

Флаги состояния отражают результат выполнения арифметических инструкций:

Бит	Обозначение	Название	Описание

0	CF	Флаг переноса	Устанавливается при переносе из старшего бита
6	ZF	Флаг нуля	Устанавливается, если результат равен 0
7	SF	Флаг знака	Равен старшему биту результата

2.3. Команда сравнения cmp

Инструкция `cmp` сравнивает два операнда и выставляет флаги:

```assembly
cmp <операнд_1>, <операнд_2>
```

2.4. Команды условного перехода

Мнемокод	Условие	Значение флагов
je / jz	a = b	ZF = 1
jne / jnz	a ≠ b	ZF = 0
jg / jnle	a > b (со знаком)	SF = OF и ZF = 0
jl / jnge	a < b (со знаком)	SF ≠ OF
jge / jnl	a ≥ b (со знаком)	SF = OF
jle / jng	a ≤ b (со знаком)	SF ≠ OF или ZF = 1

---

3. Выполнение лабораторной работы

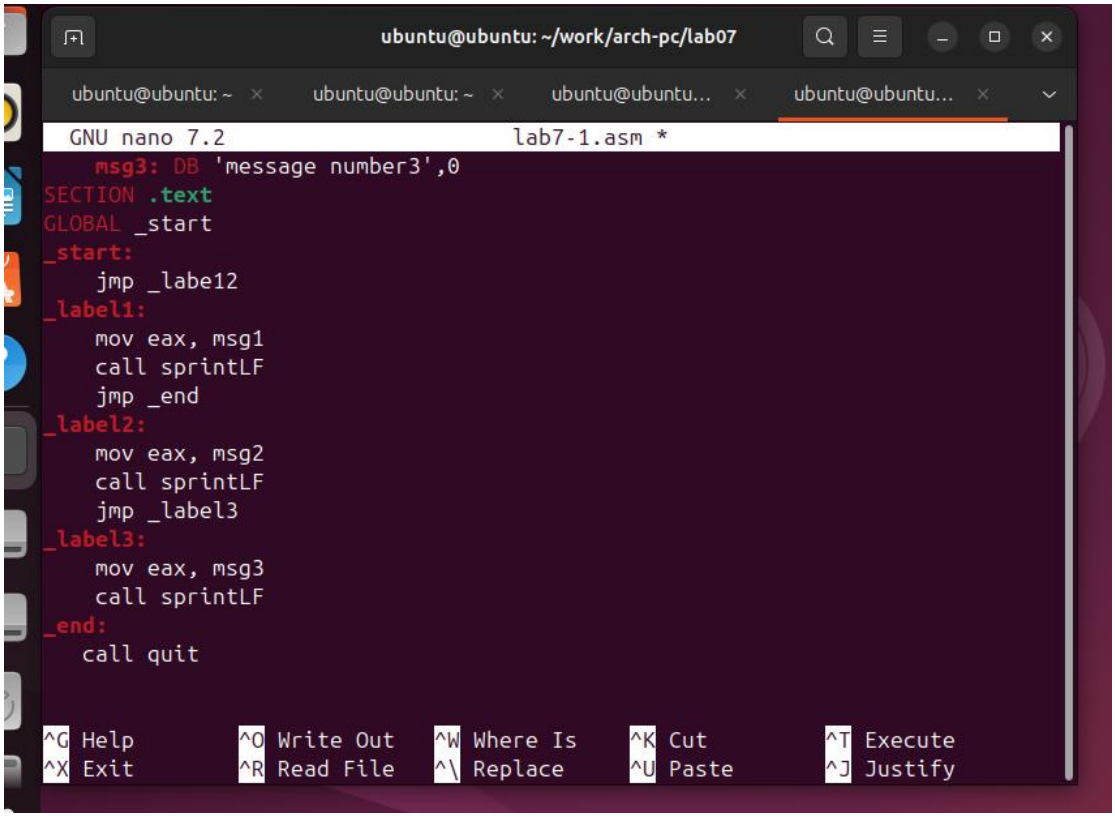
3.1. Создание рабочего каталога

Создан каталог для лабораторной работы №7 и перейдён в него:

```bash
mkdir -p ~/work/arch-pc/lab07
cd ~/work/arch-pc/lab07
```

> ** Скриншот 1:**
> *Окно терминала с созданием каталога lab07 и переходом в него

о .*



3.2. Создание и редактирование файла lab7-1.asm

Создан файл `lab7-1.asm`:

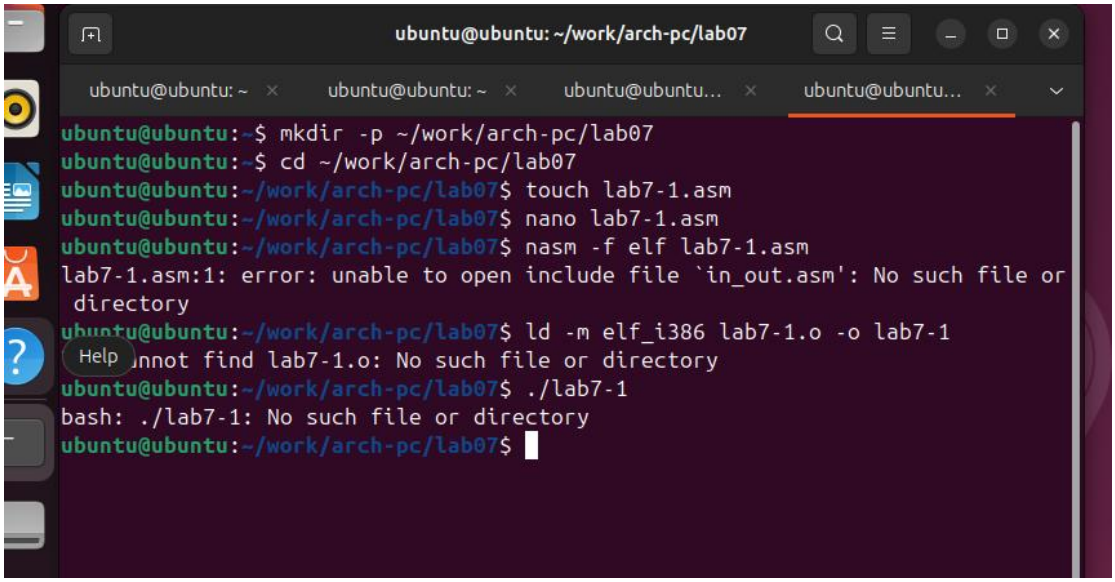
```bash
touch lab7-1.asm
nano lab7-1.asm
```

Листинг программы `lab7-1.asm`:

```assembly
%include 'in_out.asm'

SECTION .data
    msg1: DB 'Сообщение № 1', 0
    msg2: DB 'Сообщение № 2', 0
    msg3: DB 'Сообщение № 3', 0

SECTION .text
```

```
        GLOBAL _start

_start:
        jmp _label2

_label1:
        mov eax, msg1
        call sprintLF
        jmp _end

_label2:
        mov eax, msg2
        call sprintLF
        jmp _label1

_label3:
        mov eax, msg3
        call sprintLF

_end:
        call quit
```

> **Скриншот 2:**



> *Окно редактора nano с содержимым файла lab7-1.asm.*

3.3. Компиляция и выполнение lab7-1.asm

```bash
nasm -f elf lab7-1.asm
ld -m elf_i386 -o lab7-1 lab7-1.o
./lab7-1
```

Результат выполнения:
```
Сообщение № 2
Сообщение № 1
```

> ** Скриншот 3:**
> *Окно терминала с процессом компиляции и результатом выполнения программы lab7-1.asm.*

3.4. Модификация программы для вывода в порядке 3 → 2 → 1

Для изменения порядка вывода сообщений на "Сообщение № 3 → Сообщение № 2 → Сообщение № 1" внесены изменения в программу:

Модифицированная программа:

```assembly
%include 'in_out.asm'

SECTION .data
	msg1: DB 'Сообщение № 1', 0
	msg2: DB 'Сообщение № 2', 0
	msg3: DB 'Сообщение № 3', 0

SECTION .text
	GLOBAL _start

_start:
	jmp _label3

_label1:
	mov eax, msg1
```

```
        call sprintLF
        jmp _end

_label2:
        mov eax, msg2
        call sprintLF
        jmp _label1

_label3:
        mov eax, msg3
        call sprintLF
        jmp _label2

_end:
        call quit
```

Результат выполнения:

```
Сообщение № 3
Сообщение № 2
Сообщение № 1
```

3.5. Создание программы lab7-2.asm

Создан файл `lab7-2.asm` для определения наибольшего из трёх чисел:

```bash
touch lab7-2.asm
nano lab7-2.asm
```

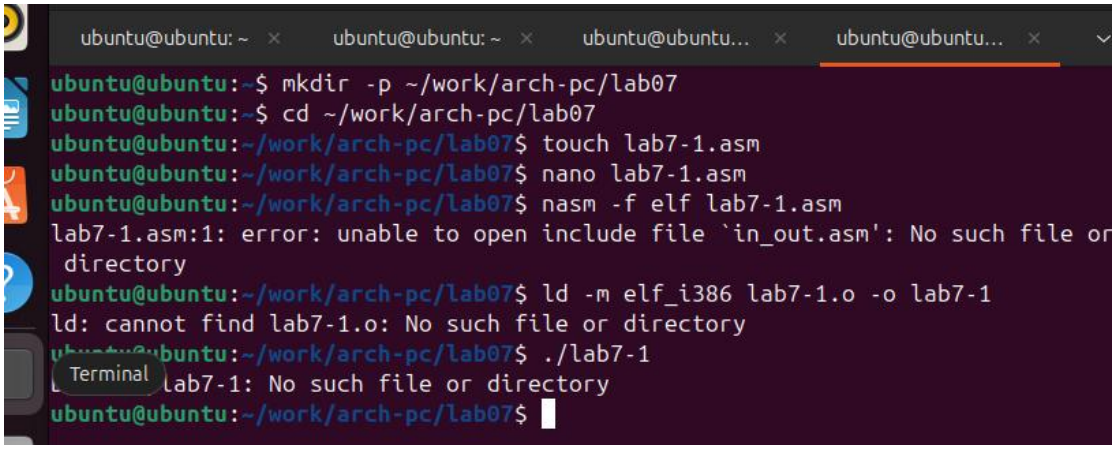
Листинг программы `lab7-2.asm`:

```assembly
%include 'in_out.asm'

SECTION .data
        msg_input:   DB 'Введите значение для B: ', 0h
        msg_result: DB 'Наибольшее число: ', 0h
```

```
        A: DD 20
        C: DD 50

SECTION .bss
        max: RESB 10
        B:      RESB 10

SECTION .text
        GLOBAL _start

_start:
        ; Вывод приглашения для ввода B
        mov eax, msg_input
        call sprint

        ; Ввод значения B
        mov ecx, B
        mov edx, 10
        call sread

        ; Преобразование ASCII в число
        mov eax, B
        call atoi

        ; Сохраняем B в переменную max
        mov [max], eax

        ; Сравнение max с A (20)
        cmp eax, [A]
        jg check_C
        mov eax, [A]
        mov [max], eax

check_C:
        ; Сравнение max с C (50)
        mov eax, [max]
        cmp eax, [C]
        jg fin
        mov eax, [C]
        mov [max], eax

fin:
        ; Вывод результата
        mov eax, msg_result
```

```
        call sprint
        mov eax, [max]
        call iprintLF
        call quit
```

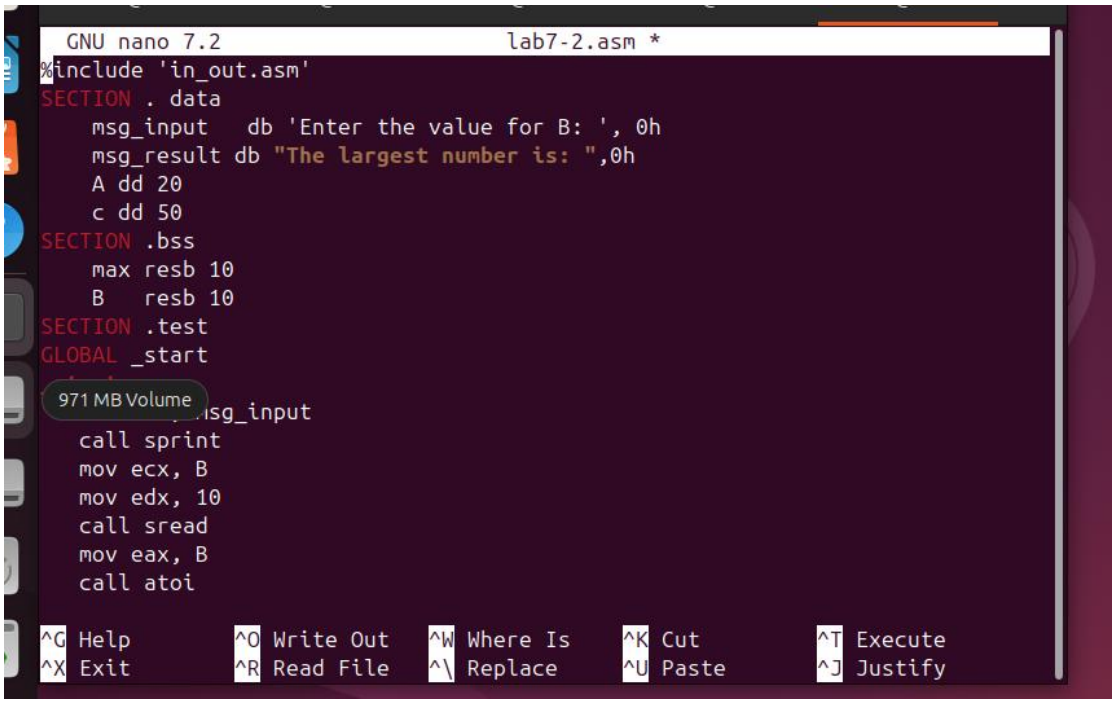
> ** Скриншот 4:**



> *Окно редактора nano с первой частью содержимого файла lab7-2.asm.*

> ** Скриншот 5:**

> *Окно редактора nano со второй частью содержимого файла lab7-2.asm.*

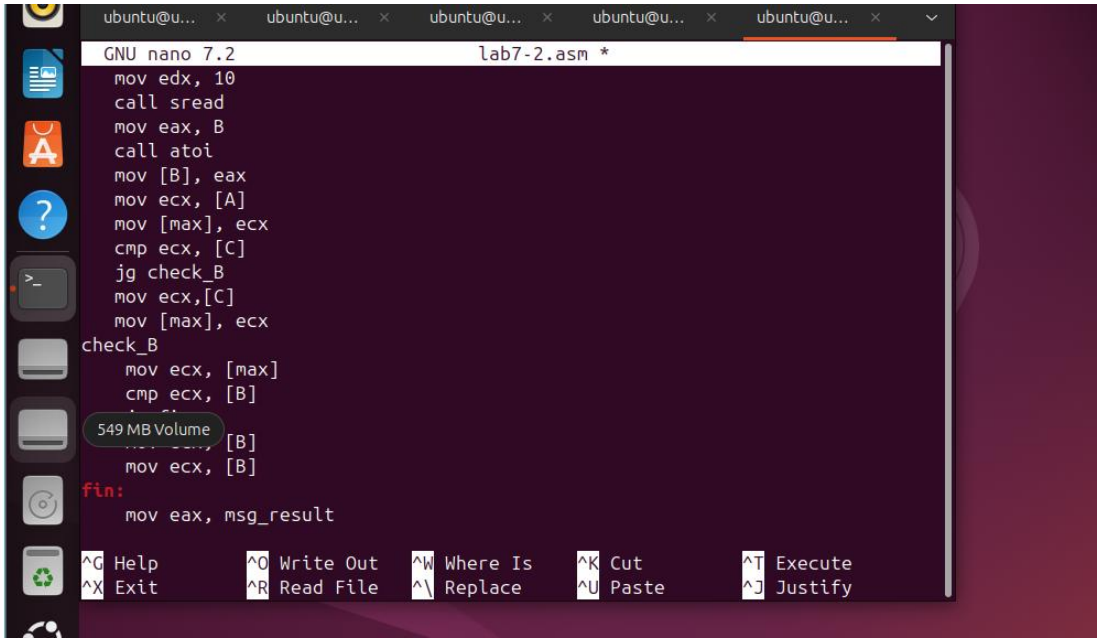


3.6. Компиляция и выполнение lab7-2.asm

```bash
nasm -f elf lab7-2.asm
ld -m elf_i386 -o lab7-2 lab7-2.o
./lab7-2
```

Результат выполнения:
```
Введите значение для B: 30
Наибольшее число: 50
```

> ** Скриншот 6:**
> *Окно терминала с процессом компиляции и результатом выполнения программы lab7-2.asm.*



3.7. Создание файла листинга

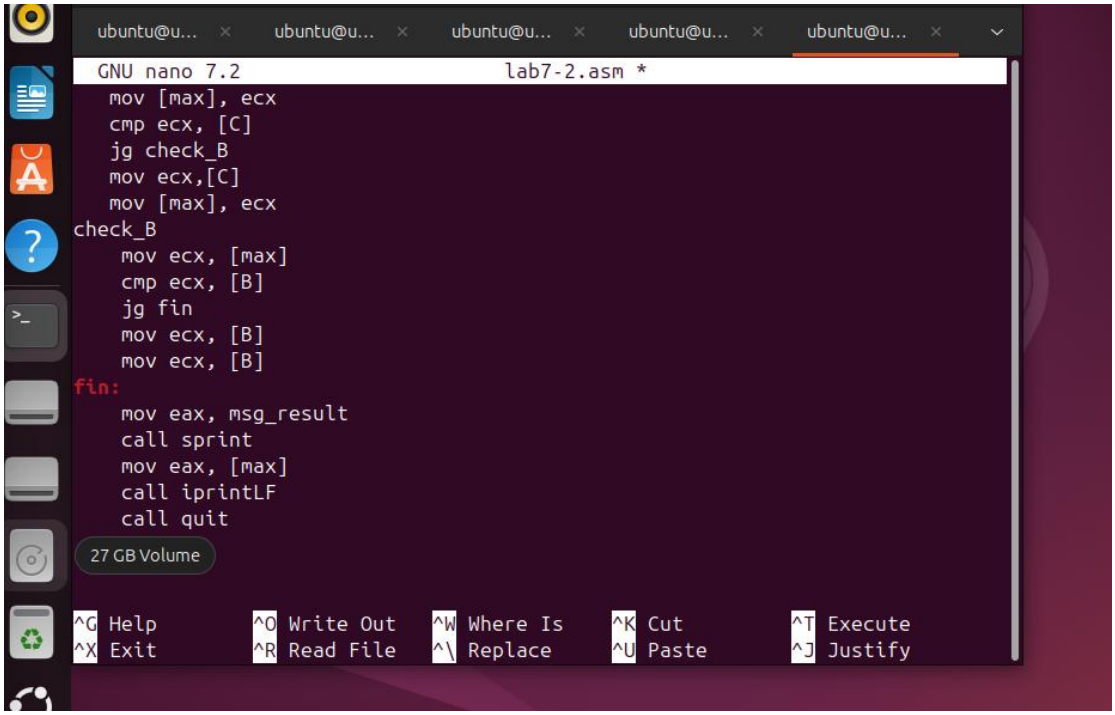
Для получения файла листинга использована команда:

```bash
nasm -f elf -l lab7-2.lst lab7-2.asm
```

Фрагмент файла листинга `lab7-2.lst`:

```
1 %include 'in_out.asm'
2
3 SECTION .data
4 00000000 4266566564... msg_input: DB 'Введите значение для B: ', 0h
5 00000016 4E61696...        msg_result: DB 'Наибольшее число: ', 0h
6 0000002B 14000000            A: DD 20
7 0000002F 32000000            C: DD 50
8
9 SECTION .bss
10 00000000 0000000000... max: RESB 10
11 0000000A 0000000000... B: RESB 10
12
13 SECTION .text
14 00000000 B8[00000000] mov eax, msg_input
15 00000005 E8(00000000) call sprint
16 0000000A B9[0A000000] mov ecx, B
17 0000000F BA0A000000      mov edx, 10
18 00000014 E8(00000000) call sread
```

> **Скриншот 7:**
> *Окно терминала с созданием файла листинга lab7-2.lst.*



---

4. Задание для самостоятельной работы

4.1. Определение варианта

По номеру студенческого билета **1132258191**:
- Вариант = (1132258191 mod 20) + 1 = 11 + 1 = **12**

4.2. Задание №1: Нахождение наименьшего из трёх чисел

Согласно таблице 7.5 для варианта 12:
- a = 21, b = 28, c = 34

Листинг программы `task7-1.asm`:

```assembly
%include 'in_out.asm'

SECTION .data
    msg_result: DB 'Наименьшее число:', 0
    a: DD 21
    b: DD 28
    c: DD 34

SECTION .bss
```

```
    min: RESB 10

SECTION .text
    GLOBAL _start

_start:
    ; Предполагаем, что a - наименьшее
    mov eax, [a]
    mov [min], eax

    ; Сравнение с b
    mov eax, [min]
    cmp eax, [b]
    jle check_c
    mov eax, [b]
    mov [min], eax

check_c:
    ; Сравнение с c
    mov eax, [min]
    cmp eax, [c]
    jle fin
    mov eax, [c]
    mov [min], eax

fin:
    ; Вывод результата
    mov eax, msg_result
    call sprint
    mov eax, [min]
    call iprintLF
    call quit
```

Результат выполнения:

```
Наименьшее число: 21
```

4.3. Задание №2: Вычисление функции f(x)

Согласно таблице 7.6 для варианта 12:

\[
f(x) =
\begin{cases}
a + 10, & x < a \\
x + 10, & x \geq a
\end{cases}
\]

Проверочные значения:
- (x_1 = 1, a_1 = 2) → 1 < 2 → f(x) = 2 + 10 = **12**
- (x_2 = 2, a_2 = 1) → 2 ⩾ 1 → f(x) = 2 + 10 = **12**

Листинг программы `task7-2.asm`:

```assembly
%include 'in_out.asm'

SECTION .data
	msg_x: DB 'Введите значение x: ', 0
	msg_a: DB 'Введите значение a: ', 0
	msg_result: DB 'Результат f(x) = ', 0

SECTION .bss
	x: RESB 10
	a: RESB 10

SECTION .text
	GLOBAL _start

_start:
	; Ввод значения x
	mov eax, msg_x
	call sprint
	mov ecx, x
	mov edx, 10
	call sread
	mov eax, x
	call atoi
	mov [x], eax

	; Ввод значения a
	mov eax, msg_a
	call sprint
```

```
        mov ecx, a
        mov edx, 10
        call sread
        mov eax, a
        call atoi
        mov [a], eax

        ; Вычисление функции
        ; f(x) = a+10, если x < a
        ; f(x) = x+10, если x ⩾ a
        mov eax, [x]
        cmp eax, [a]
        jl case1

case2:
        ; x ⩾ a
        mov eax, [x]
        add eax, 10
        jmp result

case1:
        ; x < a
        mov eax, [a]
        add eax, 10

result:
        ; Вывод результата
        mov ebx, eax
        mov eax, msg_result
        call sprint
        mov eax, ebx
        call iprintLF
        call quit
```

Проверка:
- Для x=1, a=2: результат = 2+10 = **12**
- Для x=2, a=1: результат = 2+10 = **12**

---

5. Ответы на вопросы для самопроверки

1. Для чего нужен файл листинга NASM? В чём е

го отличие от текста программы?

Файл листинга содержит дополнительную информацию: номера строк, адреса смещения машинного кода, сам машинный код в шестнадцатеричном виде. Он нужен для отладки программы.

2. Каков формат файла листинга NASM? Из каких частей он состоит?

Структура листинга:
- **Номер строки** – номер строки в файле листинга
- **Адрес** – смещение машинного кода от начала текущего сегмента
- **Машинный код** – ассемблированная исходная строка в шестнадцатеричном виде
- **Исходный текст** – исходная инструкция

3. Как в программах на ассемблере можно выполнить ветвление?

С помощью команд безусловного перехода `jmp` и команд условного перехода (`je`, `jne`, `jg`, `jl` и др.).

4. Какие существуют команды безусловного и условных переходов?

- **Безусловный:** `jmp`
- **Условные:** `je`/`jz`, `jne`/`jnz`, `jg`/`jnle`, `jl`/`jnge`, `jge`/`jnl`, `jle`/`jng`

5. Опишите работу команды сравнения cmp.

Команда `cmp` вычитает второй операнд из первого, результат никуда не записывается, но устанавливаются флаги (ZF, SF, OF, CF), которые затем используются командами условного перехода.

6. Каков синтаксис команд условного перехода?

```assembly
j<мнемокод><метка>
```

7. Приведите пример использования команды сравнения и команд условного перехода.

```assembly
cmp eax, ebx
jg label_greater
jl label_less
je label_equal
```

8. Какие флаги анализируют команды условного перехода?

Команды условного перехода анализируют флаги: ZF (нуля), SF (знака), OF (переполнения), CF (переноса).

---

6. Выводы

В ходе выполнения лабораторной работы №7:

1. **Освоены команды безусловного перехода:**
   - Использование `jmp` для изменения порядка выполнения инструкций
   - Создание программы с выводом сообщений в различных последовательностях

2. **Изучены команды условного перехода:**
   - Применение `cmp` для сравнения чисел
   - Использование `jg`, `jl`, `jle` для организации ветвлений
   - Реализация алгоритма нахождения наибольшего числа

3. **Освоена работа с файлом листинга:**
   - Создание листинга с ключом `-l`
   - Анализ структуры листинга (номер строки, адрес, машинный код, исходный текст)

4. **Выполнены самостоятельные задания:**
   - Программа для нахождения наименьшего из

трёх чисел (вариант 12: 21, 28, 34)

- Программа для вычисления функции $f(x) = \begin{cases} a+10, & x < a \\ x+10, & x \geq a \end{cases}$

Цель работы достигнута.

---

7. Приложение

Файлы, созданные в ходе работы:

- `lab7-1.asm` – программа с безусловными переходами
- `lab7-2.asm` – программа нахождения наибольшего числа
- `lab7-2.lst` – файл листинга
- `task7-1.asm` – программа нахождения наименьшего числа
- `task7-2.asm` – программа вычисления функции
- `in_out.asm` – подключаемый файл